# Il punteggio esatto

Tentativo 18: prevedere il punteggio NewsGuard (0–100) di una testata dai suoi articoli, senza classi

John Bianchi

IMT Lucca & IIT-CNR Pisa

# La domanda e i dubbi da togliere

**La domanda**: dai soli articoli di una testata mai vista, quanto bene si prevede il suo **punteggio NewsGuard**, come numero da 0 a 100? Niente classi né livelli, in nessun punto.

**I dubbi sulla regressione fatta finora** (tentativo 16):

1. i fold erano stratificati sui **5 livelli**: il punteggio **dentro** un livello era lasciato al caso;
2. l'addestramento è per articolo e le testate hanno da 10 a 201 articoli: **chi scrive di più pesa di più**;
3. la valutazione passava per le classi (quote del training): non si misurava mai **di quanti punti** si sbaglia;
4. non sapevamo se prevedere il numero serve davvero, rispetto a un classificatore.

# 1. I dati

lo stesso dataset del tentativo 16

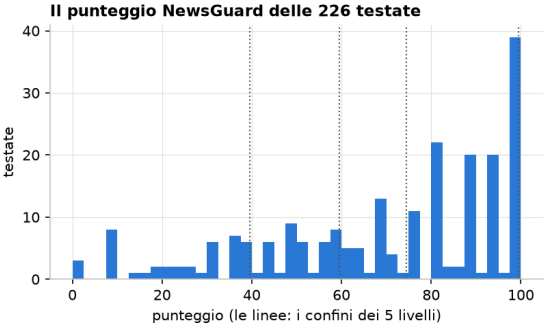
# Il dataset, passo per passo

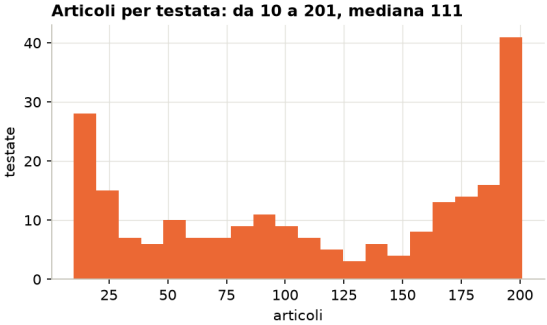
1. Testate con NewsGuard 2026 e un solo argomento, “notizie o commenti politici”; lo scraping di ottobre 2026.
2. Il testo di ogni articolo riestratto dalla pagina HTML con trafilatura in modalità precisione.
3. Solo inglese, **da 100 a 1.500 parole** (ogni articolo entra intero in 2.048 token), pubblicati dal 2025.
4. Doppioni e quasi doppioni tolti tra tutte le testate (si tiene la copia più vecchia).
5. Almeno 10 articoli per testata; **una sola testata per gruppo** di proprietà (Owner o Parent Domain di NewsGuard).
6. A ogni articolo il punteggio della sua testata.

## 226 testate, 25.524 articoli

Le etichette sono **per testata**: tutti gli articoli di una testata hanno lo stesso punteggio.

# Il punteggio e gli articoli



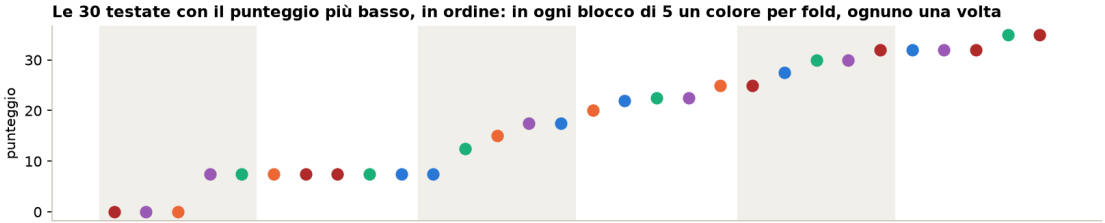




- Il punteggio è la somma dei punti di 9 criteri: solo **45 valori diversi**; 39 testate a 100, 3 a 0; media 68,5.
- Gli articoli per testata vanno da 10 a 201: le testate più prolifiche hanno 20 volte gli articoli delle altre.

# 2. Gli split

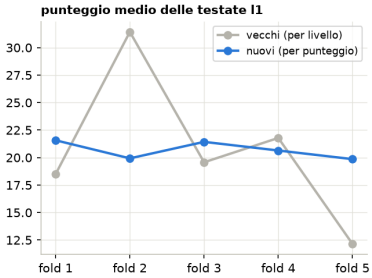
stratificati sul punteggio, non sui livelli

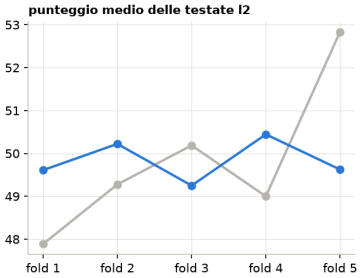
# Come si fa lo split nuovo

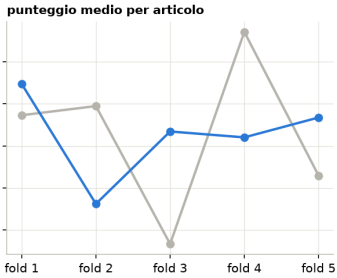



1. Le 226 testate si ordinano per punteggio (a parità di punteggio, in ordine casuale).
2. A blocchi di 5 consecutive, ogni testata va in un fold diverso (un colore per fold).
3. Ogni fold riceve così testate di tutti i punteggi, anche dentro lo stesso livello.
4. Si ripete con **3 semi diversi**: tre split, per misurare quanto i risultati dipendono dallo split.

# Split vecchi e nuovi



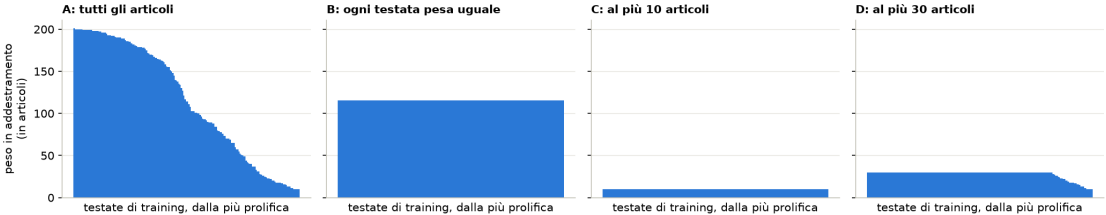




- **Dentro i livelli**: con i vecchi split le testate l1 di un fold avevano in media da 12 a 31 punti; ora da 20 a 22.
- **Per articolo** la media cambia ancora tra i fold (da 63 a 69): dipende da quanti articoli hanno le testate di ogni fold. Questo lo affrontano le condizioni dei dati.

# 3. Le condizioni dei dati

quanto pesa ogni testata in addestramento

# Cinque condizioni




**A** tutti gli articoli, peso 1: chi scrive di più pesa di più (come finora)
**B** tutti gli articoli, peso 1/(articoli della testata): **ogni testata pesa uguale**
**C** al più **10 articoli** per testata, presi a caso (10 è il minimo del dataset)
**D** al più **30 articoli** per testata
**E** **un vettore per testata**: la media dei vettori dei suoi articoli; il modello impara sulle testate

Cambia solo l'addestramento: **il test sono sempre tutti gli articoli** delle testate di test.

# Il peso per testata: il “learning rate” giusto

- In un modello lineare (ridge, logistica) il modo corretto di dire “impara meno da questa testata” è il **peso degli esempi**: l’errore di ogni articolo conta in proporzione al suo peso.
- Condizione B: ogni articolo pesa $1/n$, dove $n$ sono gli articoli della sua testata. Una testata con 200 articoli e una con 10 contribuiscono allo stesso modo.
- Nei BERT addestrati si fa la stessa cosa nell’errore quadratico di ogni lotto: $\sum_i w_i(\hat{y}_i - y_i)^2 / \sum_i w_i$.
- C e D ottengono lo stesso effetto in un altro modo, buttando via articoli; E lo porta all’estremo: una sola riga per testata.

# 4. I modelli

chi legge il testo e chi prevede il numero

# Chi legge il testo

| Lettore | Come diventa numeri | Si addestra? |
|---|---|---|
| TF-IDF | parole e coppie di parole (100.000), pesate per rarità; vocabolario dai soli testi di training | no (solo il modello sopra) |
| Mistral-Small 24B | LLM da 24 miliardi di parametri, in 4 bit; il vettore è la media degli stati interni dei token allo strato 27 di 40 (5.120 numeri) | **no**: è congelato, un lettore |
| ModernBERT-base | 150 milioni di parametri, articoli interi (2.048 token) | **sì**, tutto |
| DeBERTa-v3-base | 184 milioni di parametri, un'architettura diversa; i primi 512 token | **sì**, tutto |

Sopra TF-IDF e Mistral: **ridge** (regressore) o **logistica** (classificatore). ModernBERT e DeBERTa hanno un neurone in uscita che è direttamente il punteggio.

# Regressore o classificatore? Come si confrontano

- **Regressore**: la ridge impara il punteggio 0–100 e dà un numero per articolo.
- **Classificatore**: la logistica impara i 5 livelli e dà 5 probabilità per articolo. Per confrontarlo sullo stesso terreno, le probabilità diventano un punteggio: la media dei punteggi medi di ogni livello nel training, pesata con le probabilità.

| livello | l1 | l2 | l3 | l4 | l5 |
|---|---|---|---|---|---|
| punteggio medio nel training (circa) | 21 | 50 | 67 | 85 | 100 |
| probabilità date dal classificatore | 0,1 | 0,2 | 0,3 | 0,3 | 0,1 |

$$\text{punteggio previsto} = 0{,}1 \cdot 21 + 0{,}2 \cdot 50 + 0{,}3 \cdot 67 + 0{,}3 \cdot 85 + 0{,}1 \cdot 100 = 67{,}7$$

Se il regressore non fa meglio di questo, prevedere il numero non aggiunge nulla.

# Come si addestrano

- **Ridge**: i pesi minimizzano l'errore quadratico pesato più una penalità (alpha) sui pesi grandi, che evita di imparare a memoria. **Logistica**: come sopra, sulla probabilità del livello giusto (penalità C).
- **La penalità si sceglie dentro il training**: le testate di training si dividono in 3 (sul punteggio), si prova ogni valore della griglia e si tiene quello con l'errore medio per testata più basso; poi si riaddestra su tutto il training. Il fold di test non entra mai nella scelta.
- **ModernBERT e DeBERTa**: tutto il transformer, errore quadratico pesato sul punteggio/100, lotti di 32 articoli, learning rate 3e-5, 5 epoche, si tiene l'ultima. Solo sullo split 1 (ogni corsa costa ore): ModernBERT nelle condizioni A, B, C; DeBERTa nella B.
- **TF-IDF e Mistral**: tutte le combinazioni, **2 modelli $\times$ 2 tipi $\times$ 5 condizioni $\times$ 3 split $\times$ 5 fold = 300 addestramenti**.

# 5. La valutazione

di quanti punti si sbaglia, per articolo e per testata

# Che cosa si confronta

1. Ogni testata è nel test di un fold: **il modello non l'ha mai vista**.
2. **Articolo singolo**: il punteggio previsto per ogni articolo contro il punteggio vero della sua testata.
3. **Testata**: la media dei punteggi previsti dei suoi articoli (nella condizione E, la previsione sul vettore medio della testata) contro il punteggio vero.
4. **Riferimento**: chi prevede sempre la media delle testate di training (circa 68,5). Sbaglia in media di **22,6 punti** (RMSE 27): un modello serve solo se sta sotto.

# Le misure, con un esempio

Quattro testate:

| | testata 1 | testata 2 | testata 3 | testata 4 |
|---|---|---|---|---|
| punteggio vero | 10 | 50 | 80 | 100 |
| punteggio previsto | 30 | 55 | 70 | 85 |
| errore | $+20$ | $+5$ | $-10$ | $-15$ |

| | | |
|---|---|---|
| **MAE** | errore medio assoluto, in punti | $(20+5+10+15)/4 = 12{,}5$ |
| **RMSE** | radice dell'errore quadratico medio: pesa di più gli errori grandi | $\sqrt{(400+25+100+225)/4} = 13{,}7$ |
| **$R^2$** | quanta variabilità si spiega (0 = come dire la media, 1 = perfetto) | $1 - 750/4.600 = 0{,}84$ |
| **Spearman** | l'ordine delle testate è giusto? | 1: l'ordine è perfetto |
| **Pearson** | il legame è lineare? | 0,998 |

Nell'esempio l'ordine è perfetto ma le previsioni sono **schiacciate verso il centro**: la testata da 10 diventa 30, quella da 100 diventa 85. Il MAE lo vede, Spearman no: servono tutte e due.

# Le analisi del notebook dei risultati

1. **Il quadro**: MAE e Spearman, per testata e per articolo, di tutte le configurazioni (media dei 3 split, con il minimo e il massimo).
2. **Regressore o classificatore**: le due curve nelle cinque condizioni, per TF-IDF e Mistral.
3. **Le migliori configurazioni** con gli intervalli al 95% (bootstrap sulle testate) e il confronto con la media.
4. **Dove si sbaglia**: l'errore con il segno e l'errore assoluto per fascia del punteggio vero (0–39, 40–59, 60–74, 75–99, 100), e il grafico previsto contro vero.
5. **Il DNA in punti**: la testata giudicata dalla media di 1, 3, 5, 9, 15, 25 articoli o di tutti.
6. **Quanto conta lo split**: lo stesso modello nei tre split.

# Come leggere i risultati: le domande

| **Domanda** | **Dove si legge la risposta** |
|---|---|
| La regressione aggiunge qualcosa alla classificazione? | regressore contro classificatore trasformato in punteggio |
| Il peso delle testate prolifiche falsava l'addestramento? | condizione A contro B, C, D, E |
| Imparare sulle testate invece che sugli articoli aiuta? | condizione E contro le altre |
| Un BERT addestrato fa meglio di un lettore congelato? | ModernBERT e DeBERTa contro Mistral |
| Di quanti punti si sbaglia, e dove? | MAE contro 22,6 della media; errore per fascia |
| Quanti articoli servono per una testata? | la curva del DNA |
| I risultati dipendono dallo split? | lo stesso modello nei 3 split |

# Che cosa ci aspettiamo

- Previsioni **schiacciate verso la media**: con l'errore quadratico, un modello incerto si tiene vicino al centro. L'errore per fascia lo mostrerà (le testate peggiori sovrastimate, le migliori sottostimate).
- Il peso per testata (B) dovrebbe aiutare poco ma rendere i risultati più stabili tra gli split.
- Mistral congelato sopra TF-IDF, come nel tentativo 16; ModernBERT addestrato forse vicino a TF-IDF.
- Il DNA: l'errore per testata cala con gli articoli letti e si stabilizza dopo qualche decina.

# Dove trovare tutto

- Codice: `Code/WebScience2027/Tentativi/18_PunteggioEsatto/`
  - `01_split`: gli split; `02_lineari`: TF-IDF e Mistral (300 addestramenti, CPU);
  - `03_bert`: ModernBERT e DeBERTa (GPU); `04_confronto`: **tutti i risultati**;
  - `punteggio.py`: le funzioni (split, condizioni, modelli, misure).
- Sul server: `~/John/Tentativo18`, due code in parallelo (`t18_cpu`, `t18_gpu`); dal PC `./stato.sh` per seguire e `./scarica.sh` per scaricare.
- Gli split sono in `Dataset/tentativi/18_PunteggioEsatto/split.csv`.